

微光招新

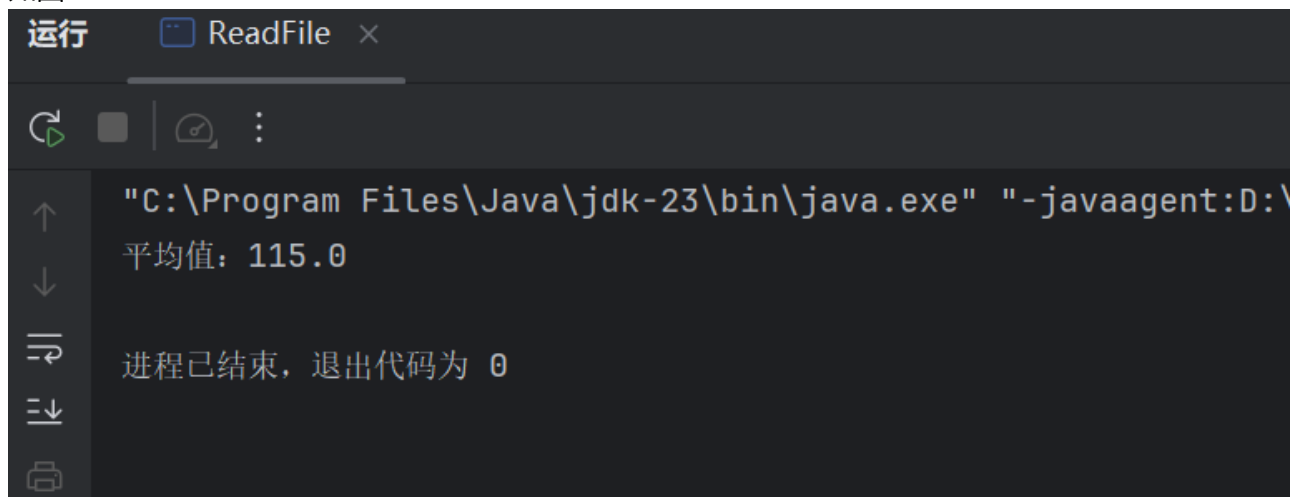
Java 07

Task1

1. Exception异常举例：一般是代码，环境问题（1）String s = null;sout(s);会抛出空指针异常。（因为s为null，s中无地址，所以会抛出空指针异常）；（2）int[] nums=new int;sout(nums[3]);会抛出数组越界异常。（因为数组长度为2，索引为0和1，3超过索引范围了，所以会抛出数组越界异常）；（3）计算100/0，会抛出算术异常。（数学逻辑不对）；（4）文件异常（文件不存在或者连接有问题）处理态度：检查代码，基于编译器或者异常类型提升修改；抛出异常（throws Exception）或者捕获异常（try catch）。Error举例:一般是系统级别的异常：（1）虚拟机错误（虚拟机故障）：（2）内存溢出（jvm没办法为对象分配足够的内存空间，比如说死循环）：线程死锁（线程互相等待）。处理态度：终止并记录程序，以便后续分析；（一般不要去捕获他）
2. (1) checkedException:1.访问不存在的变量或者方法或者文件；2.SQLException:访问数据库异常，SQL语句语法错误。3.IO异常：比如文件读取异常(2) uncheckedException:1.数学运算异常：比如除以0；2.空指针异常：比如访问null对象；3.类型转换异常：比如将字符串转换为整数。checkedException和uncheckedException区别：1. checkedException是编译时异常，编译器会检查报错；uncheckedException是运行时异常2. checkedException是继承Exception的，uncheckedException是继承RuntimeException的。

task2

3. 流程一：创建新的bank account对象，利用含参构造器初始化，Math random函数随机生成0 ~ 0.75之间的一个数字,最终得到balance变量值在0 ~ 150 之间，然后开始运行try监视器监视的可能出现异常的代码，输出第一行当前余额后，调用withdraw方法判断，由于Balanced值在[0, 150) 之间，满足判断条件，抛出自定义异常以及参数，回到main函数中，try- catch方法捕捉到异常，中断程序，运行catch后的内容，即输出"错误"+返回的输入自定义异常类含参构造器中的内容 流程二：创建新的bank account对象，利用含参构造器初始化，math.random函数随机生成0.75 ~ 1 之间的一个数，最终得到balance变量值在150 ~ 200 之间，然后，开始运行监视器中代码，输出当前余额后，调用withdraw方法，经方法中if判断，判断不成立，对balance新赋值，回到main函数中，程序，输出取款成功，跳过catch，然后输出程序结束
4. 如图：



```
运行 ReadFile x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\
平均值: 115.0
进程已结束, 退出代码为 0
```

```
ReadFile.java x NumberFormatException.java data.txt MockSongs.java

1 package com.LLY.Exceptiondemo;
2 import java.io.*;
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class ReadFile {
7     public static void main(String[] args) throws FileNotFoundException, EmptyFileException, NumberFormatException {
8         List<Integer> nums=new ArrayList<>(); //因为nums集合需要在try以外的地方使用，所以定义在try外
9         try (Reader f1=new FileReader("weiguangjava07\\src\\data.txt"); //创建一个文件输入流
10             BufferedReader br=new BufferedReader(f1) ) { //因为后边要调用br.readLine() (BufferedReader的独有方法)，这里不用多态
11             readFile("weiguangjava07\\src\\data.txt"); //自定义异常1
12             judgeemptyfile("weiguangjava07\\src\\data.txt"); //自定义异常2
13             String line; //定义一个字符串变量，用于存储读取到的一行数据，同时判断循环结束没有
14             while ((line=br.readLine())!=null) { //没有数据返回就会返回null，跟字节流返回-1不同（因为字节流是int一个len来判断读完没有）
15                 nums.add(Integer.parseInt(line)); //将字符串转换为数字，并加入集合
16             } //下面是异常处理
17             catch (IOException e){
18                 e.printStackTrace();
19             } catch (FileNotFoundException e){
20                 e.printStackTrace(); //返回异常信息
21             } catch (EmptyFileException e) {
22                 e.printStackTrace();
23             } catch (NumberFormatException e){
24                 e.printStackTrace();
25             }
26             int all=0;
27             for (int num:nums){
28                 all+=num;
29             }
30             double avg=all/nums.size();
31             System.out.println("平均值: "+avg);
32
33         }
34     }
35
36     public static void readFile(String fileName) throws FileNotFoundException { //文件不存在时抛出异常给调用者 1个用法
37         File file=new File(fileName);
38         if (!file.exists()) { //判断文件是否存在
39             throw new FileNotFoundException("文件不存在");
```

```
35
36     public static void readFile(String fileName) throws FileNotFoundException{//文件不存在时抛出异常给调用者 1个用法
37         File file=new File(fileName);
38         if (!file.exists()){//判断文件是否存在
39             throw new FileNotFoundException("文件不存在");
40         }
41     }
42
43     public static void judgeemptyfile(String fileName) throws EmptyFileException{ 1个用法
44         File file=new File(fileName);
45         if (file.length()==0){//判断文件是否为空
46             throw new EmptyFileException("文件为空");
47         }
48     }
49
50
51 }
52
53 //自定义异常1:文件不存在（IDE里面不是自带这个异常吗）
54 class FileNotFoundException extends Exception{//所有checkedException都要继承Exception类 4个用法
55     public FileNotFoundException(String message){ 1个用法
56         super(message);//调用父类的构造方法
57     }
58 }
59 //自定义异常2: 文件为空
60 class EmptyFileException extends Exception{ 4个用法
61     public EmptyFileException(String message){ 1个用法
62         super(message);
63     }
64 }
65
```

因为我已经用了try-with-resource（包含.close了），所以就没有再调用finall来关闭资源。